AUTHENTICATION HARDENING MIDDLEWARE PROXY

Van Truong Thi, Tien Truong Minh, Vinh Phan Van*

School of Computing and Information Technology, Eastern International University, Binh Duong, Vietnam

van.truong.cit23@eiu.edu.vn, tien.truong.cit23@eiu.edu.vn, vinh.phan@eiu.edu.vn

(*) Corresponding author

Abstract: Authentication endpoints serve as primary gateways to web applications, which makes them common targets for automated attacks. This project presents the design and implementation of an Authentication Hardening Middleware Proxy — a standalone Node.js reverse proxy that protects vulnerable backends without requiring changes to their source code. Targeting vulnerabilities under OWASP A07:2021 (Identification and Authentication Failures), the system uses a layered defense approach combining three main mechanisms: an IP-based sliding window rate limiter against brute force, a username-diversity tracker to detect credential stuffing, and response normalization to prevent account enumeration. The backend is also protected from timing side-channel attacks using bcrypt with a fixed work factor. The proxy is tested through automated Python scripts and a live dashboard, confirming that malicious requests are blocked at the perimeter while keeping response times consistent.

Keywords: *web security, reverse proxy, brute force defense, credential stuffing, account enumeration, rate limiting, bcrypt*

I. INTRODUCTION

In modern web applications, authentication endpoints are among the most frequently attacked surfaces. They handle identity verification directly, which makes them a constant target for automated tools and scripts. The Open Web Application Security Project (OWASP) lists identification and authentication failures as one of the top critical risks in web security [1]. A common challenge in legacy or fast-built applications is that security logic is mixed in with business code. In these cases, adding defenses like rate limiting or sanitized error messages often means touching core application logic — something that risks breaking existing functionality and complicating long-term maintenance.

Without any perimeter protection, a standard backend is exposed to three common types of credential attacks.

The first is brute force, where an attacker scripts thousands of password attempts against a single account. The second is credential stuffing — using leaked username/password pairs from other breaches to log in, which works because many users reuse passwords across sites [2]. The third is account enumeration, where small differences in error messages (like "User not found" vs. "Incorrect password") let attackers figure out which usernames actually exist before going further.

To address these without touching the backend code, this project builds an Authentication Hardening Middleware Proxy. It runs on port 4000 and sits in front of the actual application on port 3000, intercepting all POST /login requests before they reach the server.

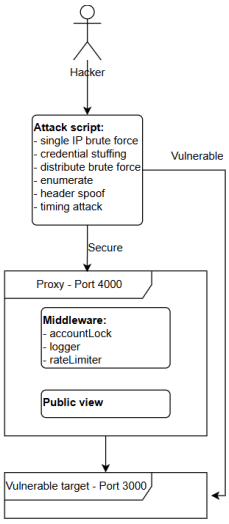

Figure 1. Basic system Flow

The proxy keeps short-lived counters in memory to track suspicious patterns, then blocks bad IPs or locks targeted accounts at the network edge. The main advantage of this setup is that the backend application needs no changes at all — the security layer is entirely separate.

II. THEORETICAL BACKGROUND

A. Authentication Vulnerabilities and Attack Vectors

Login endpoints are among the most targeted parts of a web application. Because they handle identity verification directly, they are a natural entry point for automated attacks. OWASP identifies identification and authentication failures (A07:2021) as one of the most critical security risks affecting modern web applications. Weak authentication controls can increase the likelihood of successful credential-based attacks. Three attack types are most relevant to this project.

1. Brute Force Attacks

In a brute force attack, an attacker runs a script that repeatedly submits password guesses for a single account [6]. Without any rate limiting on the backend, there is nothing stopping thousands of attempts from going through. The pattern is easy to spot in theory — many failed logins from the same IP, all targeting the same username — but without a layer that actually tracks and acts on this, the backend processes each request the same way.

2. Credential Stuffing

Credential stuffing is different from brute force in one important way: instead of guessing passwords, the attacker uses real username and password pairs leaked from other breaches. These credential datasets are often compiled from previous data breaches and may contain a large number of username-password combinations [4]. Because people often reuse passwords, a large portion of these pairs still work on other sites. The tricky part is that each account is usually only tried once or twice, so per-account lockout rules do not catch it. What gives it away is a single IP sending a high volume of login attempts spread across many different usernames.

3. Account Enumeration

Account enumeration happens when a backend returns different error messages depending on whether a username exists. For example, returning "User not found" versus "Incorrect password" tells an attacker exactly which usernames are real. By sending probes across a large list of common names or emails, an attacker can quietly build a list of valid accounts before launching a more targeted attack. The fix is straightforward - return the same generic message regardless of what actually failed - but many applications still do not do this [6].

These attack vectors were selected because they can be mitigated at the middleware layer without requiring modifications to the backend application. As a result, they provide suitable use cases for evaluating the proposed Authentication Hardening Middleware Proxy.

B. Mitigation Heuristics and Algorithms

1. Sliding Window Rate Limiting Heuristics

Rate limiting is a common defense against automated login attempts, but the way it is

implemented matters. A basic fixed-window limiter counts requests within set time intervals — say, 10 requests per minute — and resets the counter at the start of each new window. The problem is that an attacker can send 10 requests just before the reset and another 10 right after, effectively doubling the allowed rate at the boundary.

A sliding window approach avoids this by not using fixed reset points at all [7]. Instead, the proxy stores a timestamp for every incoming request from a given IP. When a new request arrives, it drops any timestamps older than the defined window $\mathbf{W_{ms}}$ and checks how many remain. If the count is still under the limit $\mathbf{N_{req}}$, the request goes through. If not, it is blocked. Formally, a request is allowed if and only if:

$$|\textbf{active_timestamps}| < \mathbf{N_{req}}$$

where active_timestamps contains only entries within the last $\mathbf{W_{ms}}$ milliseconds. Because the window moves with each request rather than resetting on a schedule, there is no boundary an attacker can exploit.

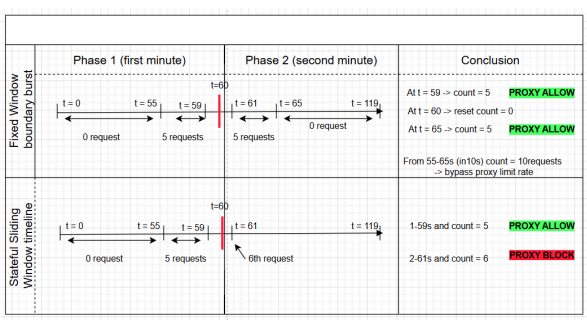

Figure 2. Comparison between a vulnerable Fixed Window boundary burst and a hardened Stateful Sliding Window timeline.

2. Timing Side-Channel Defenses

Most programming languages compare strings character by character and stop as soon as a mismatch is found. This means a comparison that fails on the first character finishes faster than one that fails on the last. In theory, an attacker can measure response times across many requests and use those differences to guess a password one character at a time.

To prevent this, passwords should never be compared as plain strings. This project uses bcrypt [2] with a cost factor of 12 to perform password verification against stored password hashes. Because bcrypt requires a computationally expensive hashing operation for every verification attempt, it significantly reduces the usefulness of timing information that might otherwise be observed during password comparisons [5].

3. Account Lockout

IP-based rate limiting works well when attacks come from a single source, but it fails when an attacker spreads requests across many IPs. Account lockout handles this by counting failed attempts at the account level instead. Once the failure count for a given username crosses a threshold, that account is temporarily locked and further attempts are rejected no matter where they come from [3].

4. Response Normalization

Some backends return different error messages depending on what went wrong — "User not found" versus "Incorrect password," for example. This difference is enough for an attacker to figure out which usernames are real, even without knowing any passwords. Response normalization fixes this by replacing all authentication failure responses with the same generic message before it reaches the client [6], so nothing about the internal state leaks out.

Together, these mechanisms provide layered protection against credential-based attacks. Sliding window rate limiting and account lockout reduce the effectiveness of automated login attempts, while response normalization and bcrypt-based password verification help mitigate information disclosure and timing-related side channels. Because all controls are implemented within the

middleware proxy, they can be deployed without modifying the backend application.

III. SYSTEM DESIGN AND METHODOLOGY

A. Architectural Overview and Data Flow

The system is built as a standalone reverse proxy that sits between clients and the backend application. The proxy runs on port 4000 while the original backend runs on port 3000, so all login traffic passes through the proxy first before anything reaches the backend.

The main benefit of this setup is that the backend does not need to be modified at all. Security controls are handled entirely at the proxy layer, which means the same approach could be applied to protect other applications without touching their code.

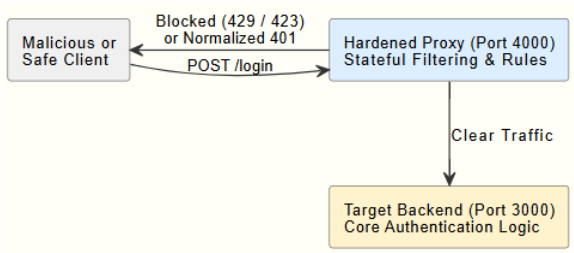

Figure 3. System architecture: proxy layer on port 4000 in front of backend on port 3000

When a request arrives at the proxy, it is evaluated against the active protection rules. If a violation is detected — such as too many requests from one IP or a locked account — the proxy returns an error response directly (HTTP 429 or 423) without forwarding the request. If the request passes all checks, it is forwarded to the backend on port 3000. Any 401 response from the backend is rewritten to a generic message before being returned to the client.

B. Compartmentalized Middleware Layers

The proxy runs three middleware stages in sequence inside **proxy/index.js** whenever a **POST /login request** comes in.

1. Dynamic IP Rate Limiting (rateLimiter.js)

This stage checks the source IP against a local cache that stores request timestamps. If an IP sends 5 or more requests within a 60-second window — formally, $|T'ip| \geq 5$ within $W = 60s$ — it is blocked for 15 minutes.

The same module also tracks username diversity per IP using a set:

$$U_{ip}=\{\text{unique usernames evaluated per IP}\}$$

If the cardinality $|U_{ip}| \geq 3$, a

CREDENTIAL_STUFFING_DETECTED flag is raised and the request is blocked before reaching the backend. This catches stuffing attacks that spread attempts across many accounts to avoid per-account lockout.

2. Account Lockout (accountLock.js)

This stage runs after the IP check and tracks failed login attempts at the account level, using req.body.username as the key. If a username accumulates 10 failed attempts from any combination of IPs, that account is locked globally for **30** minutes.

While locked, any login attempt for that username is rejected with **HTTP 423** before the request reaches the backend, regardless of where it comes from.

3. Response Normalization and Forwarding (index.js)

Requests that pass both checks are forwarded to the backend on port 3000 using Node.js's native http.request. If the backend returns HTTP 401 — whether the username does not exist or the password is wrong — the proxy replaces the response body with:

{ "error": "Invalid credentials." }

This ensures that an attacker cannot distinguish between a non-existent account and a valid account with a wrong password, mitigating account enumeration at the perimeter.

C. Shared Stateful Memory Models

The proxy stores all state in memory via proxy/store.js rather than a database. This keeps the proxy lightweight and fast — there is no I/O overhead on every request, which matters when screening high volumes of login traffic.

Table 1: In-memory state store - proxy/store.js

Map Key	Type	Stores	Expiry	Used in
`ipTimestamps`	`Map<string, number[]>`	Request timestamps per IP	Pruned after 60s	TC-01
`blacklist`	`Set<string>`	Blocked IP addresses	Auto-cleared after 15 min	TC-01, TC-02, TC-05
`accountFails`	`Map<string, number>`	Consecutive failed attempts per account	Reset on successful login	TC-04
`accountLocks`	`Map<string, object>`	Lock expiry timestamp per account	Auto-cleared after 30 min	TC-04
`ipUsernames`	`Map<string, object>`	Unique usernames tried per IP + window start	Reset after 60s	TC-02

IV. IMPLEMENTATION RESULTS

A. Test Environment Setup

All tests were run on a local machine using two independent Node.js processes simultaneously.

Table 2. Test environment software specifications

Component	Version	Role
Node.js	18+	Runtime for proxy and backend
Express	4.18.2	HTTP framework (both proxy and backend)
http-proxy-middleware	2.0.6	Passthrough proxy for non-login routes
bcrypt	6.0.0	Password hashing (target-app only)
Python	3.8+	Attack simulation scripts
pip: requests	latest	HTTP client for attack scripts

The system runs as two separate processes. The target application runs on port 3000 and handles authentication logic. The proxy runs on port 4000 and intercepts all POST /login requests before passing them to the backend. A live monitor dashboard is served by the proxy at *http://localhost:4000/dashboard* and refreshes every 2 seconds.

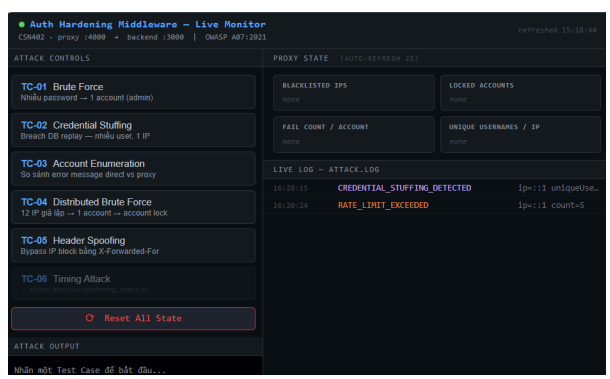

Figure 4: Proxy dashboard

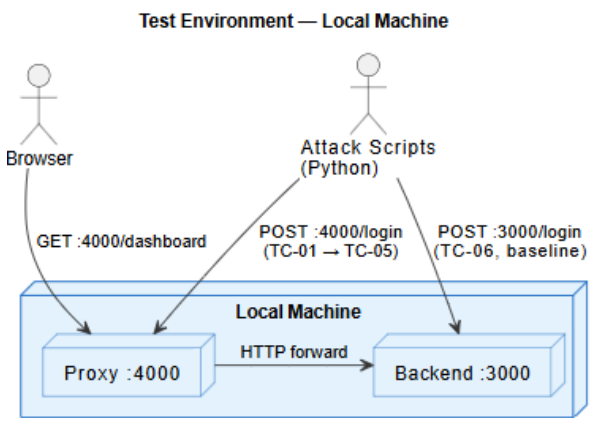

Figure 5: Local test environment topology

B. IP Rate Limiting and Brute Force Prevention

The brute force test targets the **admin** account using a 7-entry password wordlist executed by brute_force.py. The same script was run twice — once directly against the unprotected backend at port 3000, and once through the proxy at port 4000 — to produce a direct comparison.

The proxy applies a sliding window rate limiter with a threshold of 5 requests per IP per 60-second window. Once the limit is reached, the source IP is blacklisted for 15 minutes and all subsequent requests are rejected at the perimeter without reaching the backend**.**

Table 3. Brute force attack results — direct backend vs. proxy

Attempt	Password	Direct :3000	Through Proxy :4000
1	123456	401 FAIL	401 FAIL
2	password	401 FAIL	401 FAIL
3	admin	401 FAIL	401 FAIL
4	qwerty	401 FAIL	401 FAIL
5	letmein	401 FAIL	401 FAIL
6	**secret123**	**200 SUCCESS**	**429 BLOCKED**

As shown, the correct password (secret123) reached the backend on attempt 6 when no proxy was present, resulting in a successful login. Through the proxy, the same attempt was blocked before forwarding, and the correct credential was never reached to the backend. The corresponding entry recorded in attack.log is shown below:

{"type":"RATE_LIMIT_EXCEEDED","ip":"::1","count":5,"time":"..."}

{"type":"BLOCKED_BLACKLIST","ip":"::1","time":"..."}

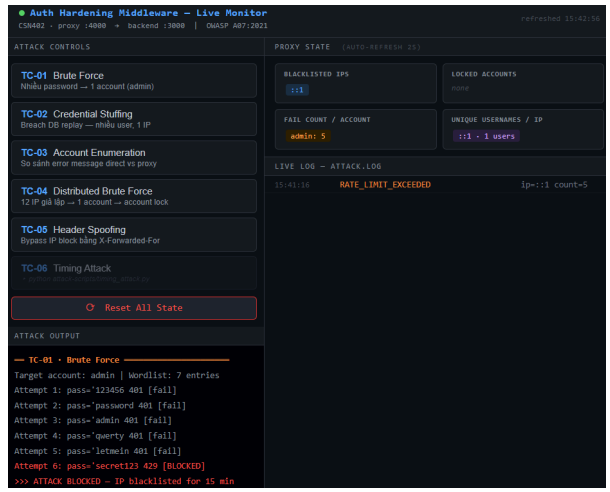

Figure 6: Proxy blocked when attempt > 5

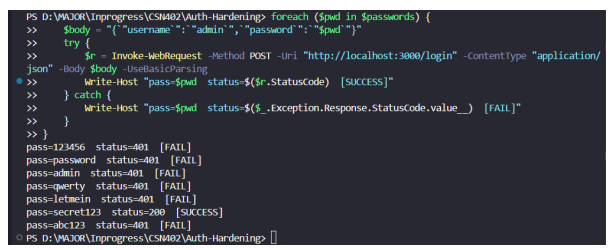

Figure 7: Direct backend allows brute force at attempt 6 (HTTP 200)

C. Credential Stuffing Detection

Unlike brute force, credential stuffing targets multiple distinct accounts with one attempt each, bypassing standard per-account lockout. The proxy detects this pattern by tracking the number of unique usernames attempted per IP within a 60-second window. Once the threshold of 3 unique usernames is reached, the IP is blacklisted before the standard rate limit (5 requests) is triggered.

Table 4. Credential stuffing detection results

Attempt	Username	Status	Unique Users from IP

01	alice@mail.com	401 miss	1
02	bob@mail.com	401 miss	2
03	carol@mail.com	**429 BLOCKED**	**3 → detected**

The attack was blocked at attempt 3 — two requests earlier than the IP rate limit would have triggered. The backend only processed 2 requests total. The corresponding attack.log entry:

{"type":"CREDENTIAL_STUFFING_DETECTED","ip":"::1","uniqueUsers":3,"accounts":["alice@mail.com","bob@mail.com","carol@mail.com"]}

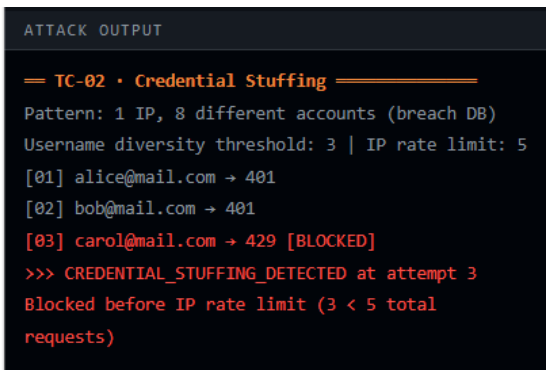

Figure 8: Credential stuffing blocked at attempt 3

D. Account Enumeration Prevention

Account enumeration exploits different error messages returned by the backend to identify valid usernames. The vulnerable backend returns distinct responses: *"User not found"* for non-existent accounts and *"Incorrect password"* for valid accounts with wrong credentials — allowing attackers to map valid usernames before launching targeted attacks.

The proxy mitigates this by normalizing all 401 responses into a single generic message, removing any distinguishing information.

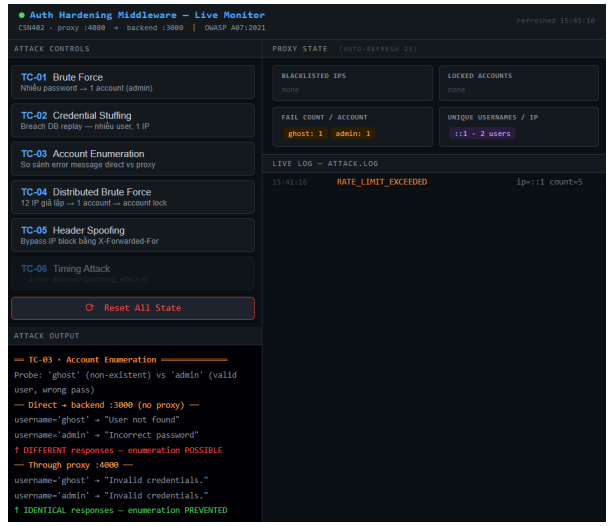

Figure 9. Output distinct backend responses when direct to port 3000

Table 5. Error response comparison — direct backend vs. proxy

Username	Condition	Direct :3000	Through Proxy :4000
`ghost`	Non-existent user	`"User not found"`	`"Invalid credentials."`
`admin`	Valid user, wrong password	`"Incorrect password"`	`"Invalid credentials."`

As shown, the direct backend exposes two distinct error messages, making enumeration easy. Through the proxy, both cases return an identical response - an attacker cannot determine whether a username exists in the system.

E. Distributed Brute Force and Account Lockout

When attacks come from multiple IPs, IP-based rate limiting alone is not enough - no single source sends enough requests to hit the threshold. TC-04 simulates this by using 12 different IPs, each sending one failed login attempt against the admin account. The proxy tracks failed attempts at the account level across all sources, and locks the account after 10 total failures (HTTP 423),

```
============================================================
  DISTRIBUTED BRUTE FORCE — target: http://localhost:3000/login
  Account: admin  |  Simulated IPs: 12
  Each IP sends 1 request — IP rate limit never triggers
============================================================

IP 10.0.0.1     pass=wrong1     status=401  [fail]  Incorrect password
IP 10.0.0.2     pass=wrong2     status=401  [fail]  Incorrect password
IP 10.0.0.3     pass=wrong3     status=401  [fail]  Incorrect password
IP 10.0.0.4     pass=wrong4     status=401  [fail]  Incorrect password
IP 10.0.0.5     pass=wrong5     status=401  [fail]  Incorrect password
IP 10.0.0.6     pass=wrong6     status=401  [fail]  Incorrect password
IP 10.0.0.7     pass=wrong7     status=401  [fail]  Incorrect password
IP 10.0.0.8     pass=wrong8     status=401  [fail]  Incorrect password
IP 10.0.0.9     pass=wrong9     status=401  [fail]  Incorrect password
IP 10.0.0.10    pass=wrong10    status=401  [fail]  Incorrect password
IP 10.0.0.11    pass=wrong11    status=401  [fail]  Incorrect password
IP 10.0.0.12    pass=wrong12    status=401  [fail]  Incorrect password
```

Figure 11. Direct backend allows unlimited failed attempts - no account lockout

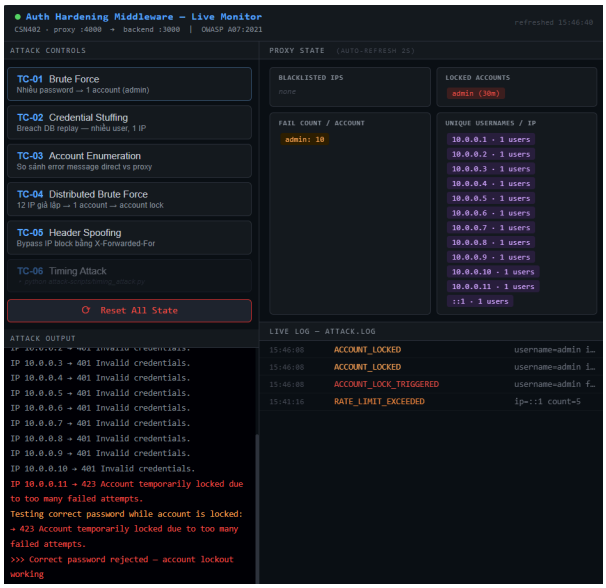

Figure 12. Account locked at attempt 10, HTTP 423

TC-05 tests whether the IP extraction logic can be bypassed using spoofed headers. After an IP is blacklisted in TC-01, the attacker injects fake values into the X-Forwarded-For header to appear as a clean IP. The proxy ignores X-Forwarded-For entirely and only uses the real socket address to identify the client, so all bypass attempts were rejected with HTTP 429.

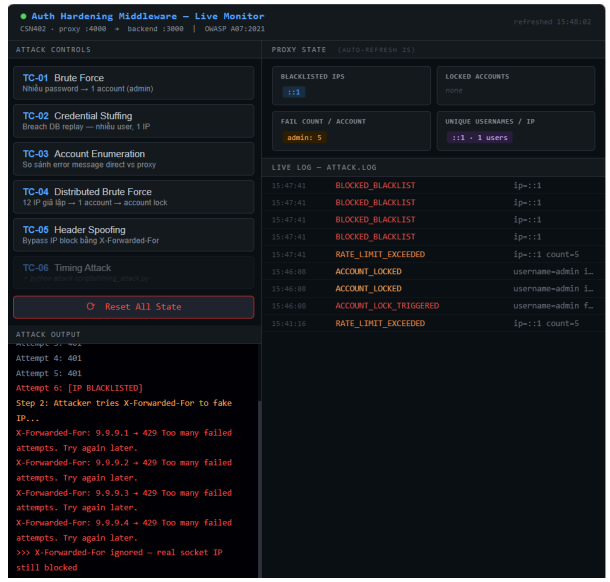

Figure 13. X-Forwarded-For is ignored by the proxy

F. Timing Attack Mitigation — bcrypt

Standard string comparison in JavaScript uses ===, which checks characters one by one and stops as soon as it finds a mismatch. This means a password that matches the first few characters takes slightly longer to reject than one that fails immediately. In theory, an attacker who measures response times across thousands of requests can use those small differences to guess the correct password one character at a time.

The backend prevents this by using bcrypt.compare() with 12 rounds instead of plain string comparison. Bcrypt always runs the full computation regardless of how early the input differs from the stored hash, so the response time stays consistent no matter what password is submitted.

```
================================================================
  TIMING ATTACK DEMO — target: http://localhost:3000/login
  Account: admin  |  Samples per category: 40 (trimmed mean)
================================================================

  Password type                          Avg (ms)    Stdev
  ------------------------------------ ---------  -------
  no overlap     'xxxxxxxx'                231.59     8.75
  2-char match   'sexxxxxx'                234.50     6.93
  5-char match   'secrexxx'                235.85     9.29
  8-char match   'secret12'                236.04     9.50
  9-char match   'secret123x'              230.90     8.38

  Timing spread across wrong passwords: 5.14 ms
```

Figure 14. timing_attack.py output - uniform ~231–236ms across all password categories

The absence of any increasing trend across prefix lengths confirms that bcrypt.compare() successfully eliminates the timing side channel.

V. CONCLUSION AND FUTURE WORK

A. Concluding Remarks

This project designed and evaluated a lightweight reverse proxy middleware layer to demonstrate practical mitigations against five authentication attack vectors identified in OWASP A07:2021. Rather than modifying the vulnerable backend,all defensive logic is handled within the proxy layer at port 4000, leaving the backend on port 3000 unchanged.

Six test cases confirmed that the implemented mechanisms behave as intended under controlled local conditions: IP-based sliding window rate limiting blocked brute-force attempts at the perimeter; username diversity detection identified credential stuffing patterns before the IP rate limit triggered; response normalization eliminated observable differences between invalid username and invalid password replies; per-account failure tracking suppressed distributed login spraying across multiple simulated source IPs; header spoofing via X-Forwarded-For was shown to have no bypass effect; and bcrypt with 12 rounds produced a timing spread too small to exploit.

These results hold within the scope of a single-machine, in-memory demonstration. The proxy does not claim to be production-hardened - state is lost on restart, and the backend remains inherently vulnerable if accessed directly. The main contribution is showing that a non-intrusive proxy layer is a practical, low-friction approach to hardening legacy authentication endpoints.

B. Future Work

The current proxy works well as a single-instance, in-memory system, but there are several directions it could be extended for larger or more demanding deployments.

1. Persistent Shared State

Right now, all state — IP blacklists, failure counts, username sets — is stored in memory and lost when the proxy restarts. For a production setup running multiple instances, this would not work since each instance would have its own separate counters. Moving state to an external store like Redis would solve both problems: state survives restarts, and multiple proxy instances can share the same data.

2. Adaptive Thresholds with Anomaly Detection

The current thresholds are fixed — 5 requests per minute, 3 unique usernames per IP. These work well in general but could be too strict or too loose depending on the situation. A future version could adjust limits dynamically based on observed traffic patterns, flagging unusual behavior rather than relying purely on hard-coded numbers.

3. CAPTCHA as a Middle Step

Currently, suspicious requests are blocked outright with HTTP 429. In some cases this could affect real users who happen to trigger a threshold. Adding a CAPTCHA challenge as an intermediate step — rather than an immediate block — would reduce false positives while still slowing down automated attacks.

REFERENCES

[1] OWASP Foundation, "A07:2021 – Identification and Authentication Failures," *OWASP Top 10:2021*, 2021. [Online]. Available: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

[2] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in *Proceedings of the 1999 USENIX Annual Technical Conference*, Monterey, CA, USA, 1999, pp. 81–92.

[3] National Institute of Standards and Technology (NIST), "Digital Identity Guidelines: Authentication and Lifecycle Management," *NIST Special Publication 800-63B*, Jun. 2017.

[4] OWASP Foundation, "Credential Stuffing Prevention Cheat Sheet," *OWASP Cheat Sheet Series*, 2023.

[5] D. Brumley and D. Boneh, "Remote Timing Attacks are Practical," *Computer Networks*, vol. 48, no. 5, pp. 701-716, 2005.

[6] OWASP Foundation, "Authentication Cheat Sheet," *OWASP Cheat Sheet Series*, 2023.

[7] N. Madden, *API Security in Action*. Shelter Island, NY: Manning Publications, 2020.